PRD: [Working Name] — Intercity Dikki-Space Logistics & Carpooling Platform

1. Problem Statement

India has fast, reliable **intra-city** quick-commerce (Zepto, Blinkit, Porter) but **inter-city** logistics is slow, expensive, and courier-dependent (2–5 days via Delhivery/DTDC for small parcels).

Meanwhile, crores of private vehicles travel between cities every day with **empty dikki (boot) space** — a wasted asset. This app connects people who want to send a parcel intercity with travelers who already have spare boot space on that route, making delivery same-day/next-day and cheaper than traditional courier.

The same vehicle network can also carry **passengers** (informal intercity carpooling), giving the platform two revenue lines on one driver-acquisition funnel.

---

2. Goals

Goal	Metric
Prove P2P + hub model works on one corridor	Bangalore ↔ Mysore live in 60 days
Build trust fast (new category, unknown drivers)	>95% verified driver KYC completion before first trip
Keep drop-off friction low for senders	Parcel booked-to-picked-up < 30 min in P2P mode
Hub model scales without booking friction	Hub drop-to-dispatch < 4 hrs

---

3. User Personas

1. **Sender/Customer** — wants to send a parcel from City A to City B, cheaper/faster than courier.
2. **Receiver** — collects the parcel, either from traveler directly (P2P) or from destination hub.
3. **Traveler/Driver** — already driving intercity (personal or for-hire), wants to earn from spare dikki space and/or empty passenger seats.
4. **Hub Manager** — runs a small physical drop point (highway-side shop, petrol pump partner, kirana

store) where parcels are staged between traveler handoffs.

5. **Passenger** — wants an intercity ride, books a seat in a traveler's car.

---

4. Core Execution Models

Model A — P2P (Traveler-to-Traveler Direct)

Sender posts a parcel + route → matched traveler picks up directly from sender's location → drops directly at receiver's location in destination city.

- Pros: fastest, no hub overhead
- Cons: traveler must deviate route slightly for pickup/drop; harder to scale trust without a hub buffer

Model B — Hub-to-Hub

Sender drops parcel at nearest hub (e.g., Bangalore hub) → any traveler heading toward Mysore picks it up from the hub (no need to meet sender) → drops at Mysore hub → receiver collects from Mysore hub.

- Pros: decouples traveler schedule from sender/receiver schedule, scales better, easier liability handoff points

- Cons: needs physical hub partners, receiver has to travel to hub unless last-mile is added later

Model C — Passenger Pooling

Same traveler profile, same trip — offers 1–3 empty seats. Independent booking flow but shares driver KYC, trip creation, and route-matching engine with Models A/B.

Recommendation: Launch with **Hub-to-Hub as the primary model** for MVP (Bangalore ↔ Mysore highway corridor) since it's operationally simpler to control and de-risks trust/liability. Layer in P2P once hub density is proven. Passenger pooling can launch in parallel since it reuses the same driver base and matching engine — it's a low marginal-cost add-on.

---

5. Core User Flows

5.1 Sender Flow (Hub Model)

1. Open app → enter pickup city/hub, drop city/hub, parcel details (weight, dimensions, category, declared value)
2. App shows price estimate + nearest hub + hub operating hours
3. Sender drops parcel at hub → hub manager weighs/scans/photographs parcel → generates a

Parcel ID + QR code

4. Sender gets live status updates: Dropped at Hub → Picked by Traveler → In Transit → Reached Destination Hub → Collected by Receiver

5.2 Traveler Flow

1. Traveler registers, completes KYC (see §7)
2. Before/during a trip: declares route (From → To), available dikki space (kg/liters equivalent or S/M/L slots), available passenger seats
3. App auto-matches parcels waiting at hubs along the route
4. Traveler visits hub → scans parcel QR → app confirms pickup (OTP-linked, see §6) → parcel added to their manifest
5. On reaching destination hub → scans/drops parcel → hub manager confirms receipt
6. Traveler earns payout (parcel fee share + any passenger fare)

5.3 Hub Manager Flow

Covered in detail in §8.

5.4 Passenger Flow

1. Search ride by route + date/time

2. See matched trips with traveler rating, vehicle type, price, seats left
3. Book seat → pay → get traveler contact + pickup point (only after booking, for safety)
4. OTP-based boarding confirmation (driver enters passenger's OTP at pickup)
5. Trip tracking shared with an emergency contact (SOS-lite for safety)

---

6. OTP Strategy

Recommendation: **use OTP only at physical custody-transfer points** — this is what protects you legally and builds sender trust, without over-friction. For the Hub-to-Hub model that's **4 checkpoints**; P2P has 2.

Checkpoint	Who enters OTP	Who generates it	Purpose
1. Sender drop at Origin Hub	Hub manager enters OTP shown on sender's app	Sent to sender's phone on parcel booking	Confirms hub received exactly this parcel from this sender

Checkpoint	Who enters OTP	Who generates it	Purpose
2. Traveler pickup from Origin Hub	Traveler enters OTP shown on hub manager's dashboard	Generated by hub manager's app when traveler arrives	Confirms custody moved from hub → traveler, timestamps liability shift
3. Traveler drop at Destination Hub	Hub manager enters OTP shown on traveler's app	Generated when traveler marks "reached destination hub"	Confirms custody moved from traveler → hub
4. Receiver pickup from Destination Hub	Hub manager enters OTP given by receiver	Sent to receiver's phone once parcel is logged at destination hub	Confirms final delivery, closes the loop, protects against wrong pickup

For **P2P mode**: only checkpoints equivalent to #2 (sender→traveler) and #4 (traveler→receiver) apply — 2 OTPs total.

For **Passenger boarding**: 1 OTP at pickup (passenger shows OTP, driver enters it in app) — keep it single-step, don't OTP the drop-off for passengers, it adds no safety value and annoys people.

Why not more OTPs: Every checkpoint should map to a change in legal/physical custody. OTPing intermediate steps (e.g., "parcel loaded in car" separate from "picked up from hub") adds friction without adding a real accountability boundary — resist the urge to add more just for "safety theater."

---

7. Driver (Traveler) Verification — KYC Flow

Given you're handling parcels + passengers, this needs to be stronger than a typical quick-commerce rider KYC:

1. **Mobile number verification** — OTP-based signup
2. **Aadhaar verification** — via Aadhaar-based e-KYC (UIDAI-licensed API partner, e.g. through a Karza/Digio/Signzy-type KYC vendor — you should NOT store raw Aadhaar numbers yourself; use

masked Aadhaar + XML/Offline eKYC or Aadhaar OTP-based verification through a licensed AUA/KUA partner, per UIDAI regulations)

3. **Driving License verification** — DL number + validity check via Parivahan/VAHAN API (mandatory if carrying passengers; recommended even for parcel-only travelers)
4. **Vehicle RC verification** — via VAHAN API, confirms vehicle ownership/registration matches driver
5. **Selfie + liveness check** — face-match against Aadhaar photo, prevents identity fraud/shared accounts
6. **Police verification / background check (optional but recommended for passenger-carrying drivers)** — third-party background check API, especially before scaling passenger pooling beyond pilot
7. **Bank account/UPI verification** — for payouts, also acts as an additional identity anchor

Sequencing recommendation: Aadhaar + selfie liveness gets a traveler "parcel-only" status quickly (fast onboarding). DL + RC verification unlocks "passenger-carrying" status — gate passenger features behind the extra verification so you're not blocking your logistics-only supply.

---

8. Hub Manager Module

8.1 Hub Manager Login

- Login via **mobile number + OTP** (not password — lower friction for small shop owners who are your likely hub partners)
- Role-based access: each hub manager account tied to one physical Hub ID (lat/long + address, assigned during onboarding by your ops team — hubs are not self-signup in MVP, you vet locations manually)
- Optional PIN/biometric lock on the hub app itself (shared shop device, multiple staff may use it)

8.2 Hub Manager Dashboard — Core Screens

1. **Incoming parcels queue** — parcels currently marked "en route to this hub" from senders
2. **Parcel intake** — scan/enter Parcel ID, weigh, photograph, enter OTP given by sender, confirm receipt
3. **Held inventory** — parcels physically at the hub awaiting a traveler match, with time-in-hub aging (flag anything >24 hrs)
4. **Traveler handoff** — when a traveler arrives, hub manager selects parcels going out with them, generates OTP for traveler to enter, confirms handoff
5. **Outbound to travelers** log — history of what left with whom, timestamp, traveler ID

6. **Receiver pickup** — when receiver arrives, hub manager verifies receiver's OTP, marks delivered, optional photo/signature capture
7. **Cash/COD reconciliation** (if you support COD) — daily settlement view
8. **Hub earnings** — commission/handling fee accrued per parcel, payout schedule

8.3 Hub Manager Incentive

Small per-parcel handling fee (flat ₹10–20/parcel suggested for pilot) — keep it simple, avoid % based fees at this stage since parcel values will be inconsistent and hard to verify.

---

9. Trust & Safety Layer

- **Parcel restrictions**: no cash, jewelry, liquids, illegal items, perishables (define a "prohibited items" list, sender must accept a declaration checkbox before booking)
- **Declared value cap** for MVP (e.g., max ₹5,000 declared value) to limit liability exposure while you build trust/insurance partnerships
- **Photo evidence** at every handoff (hub intake, traveler pickup, hub drop, receiver pickup) — protects all parties in disputes

- **Ratings**: senders rate travelers, travelers rate hub speed, passengers rate drivers (and vice versa)
- **SOS button** in passenger flow with live trip sharing to an emergency contact
- **Basic insurance/protection plan** for parcels (even a simple partnered micro-insurance product) — good trust signal, can be a paid add-on

---

10. Regulatory Notes to Flag for Legal Review

- **Passenger carpooling**: falls under Motor Vehicles Act carpooling exemptions in most states *only if* it's genuine cost-sharing (not profit-making commercial transport) — several states (Karnataka included) have had disputes with app-based pooling being treated as unlicensed aggregation. Structure passenger pricing as **cost-sharing (fuel + toll split)**, not a fare, especially in MVP, to stay defensible. Same issue you already scoped in your earlier Bangalore auto-pooling analysis — worth revisiting that legal work here since it's directly applicable.
- **Aadhaar handling**: you cannot store/collect raw Aadhaar numbers directly per UIDAI rules unless you're a licensed AUA/KUA — route through a licensed KYC vendor (Digio, Signzy, Karza, IDfy,

etc.), store only the vendor's verification token/masked reference.

- **Courier/logistics licensing**: parcel movement across state lines for commercial consideration may attract courier-service classification — check GST implications (RCM/forward charge) and whether you need any logistics/courier registration depending on scale.

---

11. MVP Scope (Suggested)

Corridor: Bangalore ↔ Mysore (highway, high traffic density, ~3.5 hr drive — matches your prior geographic instinct from the auto-pooling work)

In scope:

- Hub-to-hub parcel model (2 hubs to start)
- Traveler app: KYC, trip declaration, hub pickup/drop, OTP handoffs
- Sender app: book parcel, pay, track, OTP
- Hub manager web/app dashboard
- Passenger pooling (basic): route search, book seat, OTP boarding

Out of scope for MVP:

- P2P direct model

- COD payments
- Last-mile delivery beyond hub (receiver must come to hub)
- Insurance product (flag for phase 2)
- Multi-hop routing (hub A → hub B → hub C)

---

12. Open Questions for You

1. Are hub partners going to be paid shopkeepers/petrol pumps, or your own small kiosks? This changes the incentive model in §8.3.
2. Declared-value cap and liability language need a lawyer pass before launch — want me to draft the sender T&C / prohibited items list next?
3. Do you want passenger pooling live from day 1, or sequenced in after parcel-hub model is validated (lower initial verification burden if delayed)?

Ready to also build: pitch deck version of this, hub manager UI wireframe, or the OTP/verification flow as a sequence diagram — let me know which you want next.